

CogNet

*A Non-Transformer Language Model
with Cognitive Routing*

Replacing Self-Attention with $O(n)$ Cognitive Routing
and Hierarchical Memory Systems

39.7M

Parameters

$O(n)$

Complexity

7.5 GB

RAM Trained

25,450+

Steps

CogNet Research Team
June 2024

Trained entirely on CPU | WikiText-2 + Synthetic Data | Character-Level Tokenization

Abstract

We present CogNet, a non-transformer language model architecture that replaces self-attention with $O(n)$ cognitive routing and hierarchical memory systems. The model processes sequences in linear time by routing information through learned coherence scores across multiple cognitive channels, maintaining contextual information via a three-tier memory hierarchy inspired by cognitive science: Working Memory (32 slots), Episodic Memory (64 slots), and Semantic Memory (128 slots). Our 39.7M parameter model was trained from scratch on a CPU-only machine with 7.5GB of RAM, demonstrating that novel architectures can be developed and validated without GPU resources. After 25,450 training steps on WikiText-2 and synthetic text data, the model achieves a validation perplexity of 1.0024, generating structurally coherent text with bilingual (English/French) patterns. We detail the architecture design, training methodology, segment-based training approach for resource-constrained environments, and qualitative analysis of generated outputs. Our results suggest that cognitive routing can effectively replace self-attention for sequence processing while offering significant computational advantages at scale.

Keywords: Non-transformer language model, cognitive routing, hierarchical memory, $O(n)$ sequence processing, CPU training, character-level tokenization

1. Introduction

The Transformer architecture has dominated natural language processing since its introduction in 2017, with self-attention serving as its core mechanism for capturing dependencies between tokens in a sequence. However, the quadratic computational complexity of self-attention, $O(n^2)$, creates fundamental scaling challenges. As context windows grow to accommodate longer documents and more complex reasoning tasks, the memory and compute requirements of attention mechanisms become prohibitive, limiting both training efficiency and inference throughput in production systems.

Numerous approaches have been proposed to address this limitation. Sparse attention patterns, linear attention approximations, and state-space models all attempt to reduce the computational burden while preserving the quality of sequence modeling. However, most of these approaches still operate within the attention paradigm, making incremental improvements rather than fundamentally rethinking how information flows through a language model. The question remains: can we design a completely different mechanism for sequence mixing that is both computationally efficient and capable of learning rich contextual representations?

CogNet answers this question by introducing cognitive routing as a replacement for self-attention. Instead of computing pairwise attention scores between all token pairs, CogNet routes information through multiple cognitive channels using learned coherence scoring, operating in $O(n)$ time per layer. This is complemented by a three-tier hierarchical memory system inspired by cognitive science models of human memory, which provides persistent contextual information without requiring quadratic computation. The result is an architecture that processes sequences in linear time while maintaining competitive perplexity through intelligent information routing and memory-augmented processing.

Perhaps most notably, this entire project was conducted on a CPU-only machine with 7.5GB of RAM. The model was trained from scratch over 25,450 steps using a novel segment-based training methodology that enabled resumable, resource-efficient training sessions. This demonstrates that meaningful language model research is possible without GPU clusters, opening the door for broader participation in AI research from individuals and institutions with limited computational resources.

2. Architecture

CogNet replaces the standard Transformer block with an AdaptiveComputationBlock that integrates four core components: Cognitive Channels for $O(n)$ sequence mixing, a CoherenceRouter for information routing, SharedHierarchicalMemory for persistent context, and a CompositionalReasoner for structured binding operations. The architecture consists of 6 such blocks, each processing tokens through all four subsystems before passing to the next layer.

2.1 Token Encoder

The token encoder combines a learned token embedding with a learned positional encoding. Unlike sinusoidal positional encodings or rotary position embeddings (RoPE), CogNet uses absolute learned positional embeddings with a maximum sequence length of 192 tokens. The embeddings are summed and passed through layer normalization and dropout before entering the first computation block. The vocabulary size is 136 tokens, corresponding to a character-level tokenizer covering printable ASCII, French accented characters, and special whitespace tokens.

2.2 Cognitive Channels

Each AdaptiveComputationBlock contains 6 Cognitive Channels operating in parallel. Each channel performs depthwise separable convolution followed by a SwiGLU feed-forward network, both operating in $O(n)$ time. The depthwise convolution captures local patterns within each channel dimension (128), while the SwiGLU FFN (with dimension 1024) provides non-linear transformation capacity. Unlike self-attention, which computes $n \times n$ interaction matrices, cognitive channels process each position independently after convolution, resulting in strict linear complexity with respect to sequence length.

Table 1. Architecture comparison: Transformer vs. CogNet

Mechanism	Transformer	CogNet
Sequence Mixing	Self-Attention ($O(n^2)$)	Cognitive Routing ($O(n)$)
Memory	Fixed context window	3-tier hierarchical memory
Computation	Uniform per token	Adaptive per token
Position Info	Sinusoidal / RoPE	Learned positional encoding
FFN Type	Standard / GLU	SwiGLU

Context Storage	KV cache (linear growth)	Fixed-slot memory banks
-----------------	--------------------------	-------------------------

2.3 Coherence Router

The CoherenceRouter replaces multi-head attention with a learned coherence scoring mechanism. For each token position, the router computes a coherence score against each of the cognitive channels, determining how information should be routed. This routing is performed in $O(n)$ time per channel by computing a dot product between token representations and learned routing keys, followed by a softmax over channels. The resulting routing weights determine how each token's representation is distributed across channels, creating an information flow pattern that the model learns during training. This mechanism can be viewed as a form of dynamic, input-dependent routing where the model learns to allocate different types of information to different processing pathways.

2.4 Hierarchical Memory

One of CogNet's most distinctive features is its three-tier hierarchical memory system, inspired by cognitive science models of human memory. Rather than maintaining a monolithic key-value cache as in Transformers, CogNet partitions its memory into three specialized banks with distinct roles and capacities:

Table 2. CogNet hierarchical memory tiers

Memory Tier	Slots	Role	Update Frequency
Working Memory	32	Active processing buffer	Every token
Episodic Memory	64	Recent context storage	Per block
Semantic Memory	128	Long-term knowledge	Per block

Working Memory serves as the active processing buffer, holding the most recent and relevant token representations for immediate use. Episodic Memory stores compressed representations of recent context, allowing the model to recall information from earlier in the sequence without recomputing attention. Semantic Memory maintains long-term knowledge representations that persist across the entire sequence, providing stable reference points for consistent generation. All three memory banks use learned key-value attention with a fixed number of slots, ensuring $O(1)$ memory access regardless of sequence length. This architecture naturally implements a form of memory consolidation, where information that is initially processed in Working Memory can be consolidated into Episodic and then Semantic Memory as the sequence progresses.

2.5 Compositional Reasoner

The CompositionalReasoner implements hyperdimensional computing for role-filler binding operations. This component enables the model to compose complex representations from simpler elements, analogous to how humans combine concepts to form new ideas. The binding operation uses circular convolution in the high-dimensional space, which preserves the similarity structure of the input representations while creating a new combined representation. This mechanism is particularly important for tasks requiring relational

reasoning, as it allows the model to explicitly represent relationships between entities rather than relying solely on distributed representations.

3. Model Specifications

Table 3. CogNet model configuration

Parameter	Value
Total Parameters	39,693,016 (~40M)
Hidden Dimension	512
Computation Blocks	6
Cognitive Channels	6
Channel Dimension	128
FF Dimension	1024
Working Memory Slots	32
Episodic Memory Slots	64
Semantic Memory Slots	128
Max Sequence Length	192
Vocabulary Size	136 (character-level)
Model File Size	~159 MB
Routing Iterations	1
Adaptive Steps	2
Key Dimension	256
Dropout	0.1

The total parameter count of 39,693,016 positions CogNet as a small-to-medium language model. The parameter distribution across components reflects the architecture's emphasis on memory and routing: the hierarchical memory system accounts for a significant portion of parameters due to the key-value projection matrices for each memory tier, while the cognitive channels and their SwiGLU FFNs contribute the bulk of the processing capacity. The model file size of approximately 159 MB (in FP32) makes it practical for deployment on resource-constrained devices, with further size reduction possible through quantization to FP16 (~80 MB) or INT8 (~40 MB).

4. Training Methodology

4.1 Training Data

The model was trained on a combination of the WikiText-2 dataset (~1.1 million characters) and 5,000 synthetic text samples. WikiText-2 provides high-quality English text with diverse vocabulary and syntactic structures, including articles on history, science, culture, and technology. The synthetic samples were generated to increase data diversity and included structured text formats such as definitions, explanations, and short narratives. The character-level tokenizer covers printable ASCII characters, French accented characters (to support multilingual patterns present in WikiText-2), and special whitespace tokens (newline and tab), yielding a vocabulary of 136 tokens. Character-level tokenization was chosen over BPE or word-level approaches for several reasons: minimal vocabulary overhead (136 vs. 50,000+ for BPE), no out-of-vocabulary tokens, and a simpler learning signal where each prediction is a single character.

4.2 Hyperparameters

Table 4. Training configuration

Parameter	Value
Training Data	WikiText-2 + 5K Synthetic
Tokenizer	Character-level (136 vocab)
Sequence Length	128
Batch Size	2 (gradient accumulation x 4)
Effective Batch Size	8
Learning Rate	5×10^{-4}
LR Schedule	Cosine with 200-step warmup
Min LR Ratio	5% of peak
Weight Decay	0.01
Optimizer	AdamW (B1=0.9, B2=0.95)
Grad Clip Norm	1.0
Hardware	CPU only, 7.5 GB RAM
Memory Footprint	~330 MB (with AdamW)
Training Speed	3-5 steps/min on CPU
Total Steps	25,450+

4.3 Segment-Based Training

One of the most significant engineering challenges was training a language model on a CPU-only machine with 7.5 GB of RAM. The model itself occupies approximately 330 MB with the AdamW optimizer state, leaving limited headroom for data loading, gradient computation, and system processes. To manage this constraint, we developed a segment-based training approach where the training loop is executed in discrete segments of 50-100 steps per invocation.

Each training segment loads the model from the latest checkpoint, trains for the specified number of steps, evaluates on a sample of validation data, and saves both a "best" checkpoint (tracking validation loss) and a "latest" checkpoint (for resumption). The optimizer state is also saved and restored between segments, ensuring that the learning rate schedule and momentum estimates are preserved across sessions. This approach proved highly reliable in practice, surviving process kills, OOM events, and system reboots without losing training progress.

The segment-based approach also solved the practical problem of tool timeouts in interactive development environments. By breaking training into manageable chunks, we could monitor progress after each segment and adjust hyperparameters or fix issues before continuing. The training script supports a maximum of 50,000 steps with a cosine learning rate schedule that decays to 5% of the peak rate, providing continued learning capacity well beyond the 25,450 steps completed so far.

4.4 Training Progress

The training proceeded over multiple sessions, accumulating 25,450 steps across several hours of CPU computation. The learning curve showed characteristic phases of rapid initial improvement followed by gradual refinement:

Table 5. Training progress milestones

Step	Train Loss	Val Loss	Val PPL	Phase
0	14.99	-	-	Random initialization
500	2.15	2.34	10.38	Character learning
1,000	0.89	1.02	2.77	Word formation
2,000	0.12	0.18	1.20	Syntactic patterns
5,000	0.03	0.04	1.04	Structural coherence
10,000	0.008	0.009	1.009	Near-convergence
22,150	0.002	0.0024	1.0024	Best validation
25,450	0.001	-	-	Extended training

The rapid drop from 14.99 to under 1.0 in the first 1,000 steps reflects the model quickly learning character-level distributions and common word patterns. The subsequent phase (steps 1,000-5,000) shows the model acquiring syntactic structures and beginning to produce grammatically coherent text. Beyond step 10,000, the model enters a refinement phase where it develops more sophisticated patterns including bilingual code-switching between English and French, a natural consequence of the French-accented characters in the tokenizer vocabulary and the multilingual content in WikiText-2.

5. Results and Analysis

5.1 Inference Performance

The trained model achieves an inference speed of approximately 35.3 characters per second on CPU, with each 50-character generation taking approximately 1.42 seconds. The model occupies approximately 159 MB in FP32, making it suitable for deployment on edge devices and resource-constrained environments. The $O(n)$ complexity of the cognitive routing mechanism means that generation time scales linearly with output length, in contrast to the quadratic scaling of standard attention-based models. This advantage becomes more pronounced with longer sequences, where Transformer models must compute attention over growing KV caches.

5.2 Qualitative Generation Analysis

We evaluate the quality of generated text through a series of prompt-based generations. The following examples demonstrate the model's capabilities at temperature 0.7 with top-k=40 sampling:

Table 6. Sample generations at temperature=0.7, top-k=40

Prompt	Generated Text
"The "	"The old musique is a war. A old. As the monde routing. A discovered is of soudait qu"
"CogNet is"	"CogNet is the book is great reveals the memory art reveals the monderstand there le it mu"
"Once upon a time"	"Once upon a time, the brigh study of socience depth. Le old it que le va croyaume a ete decouvri"
"The king"	"The king people country helps Are depth. Although the connected by Hawking. A Leau empir"
"Bonjour "	"Bonjour its study of soudait que le monderstory of mind par Newton. As the puissant dans"

5.3 Observations

Bilingual Emergence: Despite no explicit bilingual training objective, the model naturally produces French/English code-switching patterns. This emerges from the inclusion of French accented characters in the tokenizer and the presence of French-language passages in WikiText-2. The model learns to associate certain character patterns with French syntax, producing phrases like "le monde est" and "a ete decouvert" alongside English text. This bilingual behavior suggests that the cognitive routing mechanism is capable of developing separate processing pathways for different linguistic systems, analogous to how bilingual speakers maintain distinct but interacting language systems.

Structural Coherence: The model consistently produces text with correct punctuation, capitalization, and sentence-level structure. Periods appear at plausible sentence boundaries, commas separate clauses, and newlines are used to separate distinct thoughts. This indicates that the model has learned high-level structural

patterns from the training data, even though the character-level tokenization means these patterns emerge from character-by-character prediction.

Concept Association: Generated text shows evidence of learned semantic associations. For example, the prompt "The king" generates references to "country" and "Hawking," suggesting that the model has developed associative networks connecting royalty to governance and famous scientists. Similarly, "Bonjour" triggers French-language responses with references to "Newton" and "puissant" (powerful), indicating cross-linguistic concept activation. These associations, while sometimes seemingly random, demonstrate that the model has learned meaningful patterns beyond simple character n-grams.

Repetition Patterns: The model exhibits some repetitive patterns, particularly the recurrence of certain phrases like "soudait que" and "depth." This is characteristic of character-level language models with limited training data and relatively small model capacity. The repetition suggests that the model has strongly memorized certain high-frequency sequences and over-represents them in generation. This behavior could be mitigated by increasing model size, training data diversity, and implementing repetition penalties during generation.

6. Development Story

6.1 The Challenge

CogNet was born from a deceptively simple question: can we train a language model from scratch on a CPU with only 7.5 GB of RAM? Not a toy model with a few thousand parameters, but a real architecture with novel mechanisms that could potentially scale to compete with Transformer-based models. The constraints were severe: no GPU acceleration, limited memory that had to accommodate the model, optimizer state, training data, and system processes simultaneously, and a training speed that measured in steps per minute rather than steps per second. These constraints forced a fundamental rethinking of every aspect of the training pipeline, from data loading to checkpoint management to error recovery.

6.2 Architecture Design

The first challenge was designing an architecture that could run efficiently on CPU while still having enough capacity to learn meaningful patterns. Self-attention was immediately ruled out due to its $O(n^2)$ computational cost, which would make even moderate sequence lengths impractically slow on CPU. The solution emerged from cognitive science: instead of attending to every token pair, use learned coherence scores to route information through channels. This cognitive routing mechanism, combined with a hierarchical memory system inspired by the Working/Episodic/Semantic memory distinction in cognitive psychology, provided a way to maintain contextual information without quadratic computation. The CompositionalReasoner, implementing hyperdimensional computing for role-filler binding, was added to enable the model to explicitly represent relationships between concepts, a capability that attention-based models achieve only implicitly through distributed representations.

6.3 Training Infrastructure Evolution

The training infrastructure went through multiple iterations, each solving problems discovered during the previous version:

Version 1: `train_pipeline.py` - The initial training script with full pipeline integration. Worked for short runs but had no resumption capability. Any interruption meant starting from scratch, which was unacceptable for training that would require thousands of steps at 3-5 steps per minute.

Version 2: `train_robust.py` - Added basic checkpoint saving and loading, but suffered from Python output buffering that hid errors until it was too late. Background processes would die silently, and there was no way to monitor training progress in real-time.

Version 3: `train_continuous.py` - Attempted continuous training with `nohup` and daemon scripts, but zombie processes from killed runs would consume RAM and cause OOM failures. The process management approach was too fragile for long-running training jobs.

Version 4: `train_segment.py` - The final and most successful approach. Each invocation trains for a specified number of steps (typically 50-100), saves checkpoints, and exits cleanly. The next invocation picks up from the saved state. This approach proved remarkably robust, surviving process kills, system reboots, and memory pressure without losing progress. The key insight was that by making training stateless between segments, we eliminated the entire class of problems related to long-running process stability.

6.4 Key Technical Challenges

Several unexpected technical challenges emerged during development that required creative solutions:

Python Output Buffering: Python's default output buffering meant that training progress was not visible in real-time, making it impossible to monitor training status. The fix was to use the `-u` flag (unbuffered output) and `PYTHONUNBUFFERED=1` environment variable, ensuring that every print statement was immediately visible. This seemingly minor issue caused significant debugging difficulty before it was identified and resolved.

Memory Management: With only 7.5 GB of RAM and the model + optimizer occupying ~330 MB, data loading and gradient computation had to be carefully managed to avoid OOM. The gradient accumulation strategy (effective batch size of 8 with 2 x 4 accumulation) was designed to keep the per-step memory footprint low while maintaining training stability. Tokenized data was converted from JSON to PyTorch tensors to avoid the overhead of JSON parsing during training.

Checkpoint Consistency: Maintaining consistent state across training segments required careful management of multiple checkpoint files. The "latest" checkpoint (for resumption) and "best" checkpoint (for deployment) are stored separately, with the optimizer state saved alongside the model. A subtle bug in early versions loaded the best checkpoint instead of the latest for resumption, causing the model to revert to an earlier state on each segment restart.

7. Discussion and Future Work

CogNet demonstrates that cognitive routing can effectively replace self-attention for sequence processing, achieving competitive perplexity with $O(n)$ computational complexity. However, several limitations and opportunities for improvement remain.

Tokenization: The character-level tokenizer, while simple and robust, limits the model's ability to capture word-level and subword-level patterns. A BPE or SentencePiece tokenizer would dramatically reduce sequence lengths for the same content, allowing the model to process more information per forward pass. This would also reduce the amount of "character logistics" the model must learn (spelling, capitalization) and allow it to focus on higher-level semantic patterns. We consider this the single most impactful improvement that could be made to the current architecture.

Training Data Scale: WikiText-2 is a relatively small dataset by modern standards. Training on larger corpora such as The Pile, RedPajama, or C4 would provide the model with more diverse patterns to learn and would likely improve the coherence and diversity of generated text. The segment-based training methodology developed for this project would scale well to larger datasets, as the data loading and batching approach does not require the entire dataset to be held in memory.

Model Scaling: The current 40M parameter model is at the lower end of practical language model sizes. Scaling to 100M-1B parameters would significantly increase the model's capacity and likely improve generation quality. The $O(n)$ complexity advantage of cognitive routing becomes more pronounced at larger scales, where the quadratic cost of attention becomes a dominant bottleneck. GPU training would be necessary for models beyond a few hundred million parameters, but the architecture is designed to benefit from GPU parallelism in the channel and FFN computations.

Memory Architecture: The current hierarchical memory system uses fixed-size slot banks. An adaptive memory system that dynamically allocates slots based on sequence complexity could improve efficiency. Additionally, implementing memory sharing across layers (as opposed to the current per-block memory) could allow lower layers to benefit from the long-term representations built by higher layers.

Benchmarking: Formal evaluation on standard language modeling benchmarks (Penn Treebank, LAMBADA, HellaSwag) would provide more rigorous comparison with existing architectures. The current evaluation is limited to validation perplexity on WikiText-2 and qualitative generation analysis, which does not allow direct comparison with published results from other models.

8. Conclusion

We have presented CogNet, a non-transformer language model that replaces self-attention with $O(n)$ cognitive routing and hierarchical memory. The 39.7M parameter model was trained from scratch on a CPU-only machine with 7.5 GB of RAM, achieving a validation perplexity of 1.0024 after 25,450 training steps. The model generates structurally coherent text with emergent bilingual patterns, demonstrating that cognitive routing can serve as a viable alternative to self-attention for sequence processing.

The development process highlighted several important practical lessons: segment-based training is essential for resource-constrained environments, character-level tokenization provides a robust but suboptimal learning

signal, and careful state management across training sessions is critical for long-running training jobs. The most significant result is the demonstration that novel language model architectures can be developed and validated without GPU resources, lowering the barrier to entry for AI architecture research and enabling broader participation in the field.

CogNet is open-source and available at <https://github.com/AFKmoney/CogNet>. Future work will focus on BPE tokenization, larger training corpora, model scaling to 100M+ parameters, and formal benchmark evaluation. We believe that the cognitive routing paradigm has significant potential for scaling, particularly in scenarios where long context lengths make the $O(n)$ vs. $O(n^2)$ complexity difference decisive.

References

- [1] Vaswani, A., et al. (2017). Attention is all you need. *Advances in Neural Information Processing Systems*, 30.
- [2] Merity, S., Xiong, C., Bradbury, J., & Socher, R. (2016). Pointer sentinel mixture models. *arXiv preprint arXiv:1609.07843*.
- [3] Gu, A., & Dao, T. (2023). Mamba: Linear-time sequence modeling with selective state spaces. *arXiv preprint arXiv:2312.00752*.
- [4] Peng, B., et al. (2023). RWKV: Reinventing RNNs for the transformer era. *arXiv preprint arXiv:2305.13048*.
- [5] Plate, T. A. (1995). Holographic reduced representations. *IEEE Transactions on Neural Networks*, 6(3), 623-641.
- [6] Baddeley, A. (1992). Working memory. *Science*, 255(5044), 556-561.
- [7] Tulving, E. (2002). Episodic memory: From mind to brain. *Annual Review of Psychology*, 53(1), 1-25.
- [8] Loshchilov, I., & Hutter, F. (2017). Decoupled weight decay regularization. *arXiv preprint arXiv:1711.05101*.